

AprilTag Machine Code Tracking (No-PTZ)

1. Experiment Objective

The camera tracks an AprilTag marker, and the car rotates to keep the Tag at the center of the frame. It supports Tracking mode (rotation only) and Follow mode (rotation + distance keeping).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
AprilTag	A visual fiducial marker system based on QR-like codes that supports 6-DoF pose estimation and is commonly used in robot localization and navigation
tag36h11	One of the AprilTag marker families, using 36-bit encoding with 11-bit error detection to provide highly robust 2D marker recognition
3D Pose Estimation	Computes the 6-DoF pose of the Tag relative to the camera from the coordinates of the Tag's 4 corner points in the image using the PnP algorithm
quad_decimate	The image downsampling factor; increasing it improves detection speed but reduces long-distance detection capability
PD Control	Proportional-derivative controller that computes the body angular velocity based on the Tag center deviation and its rate of change
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
  → april_tag_tracker_node
    └─ Image processing (15Hz): image conversion → flip →
apriltag.Detector.detect() → Tag corners + pose
    └─ Body control (10Hz): pixel error → PD angular velocity + Tag depth Z-
value distance following → /cmd_vel (BEST_EFFORT)
```

Follow mode feature: distance following is based not on pixel size but on the Z value of the Tag 3D pose (actual depth, in meters), providing higher precision.

3. Core Topic Quick Reference

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.30~0.30 m/s	☐ forward, ☐ backward
			angular.z	-0.60~0.60 rad/s	☐ turn left, ☐ turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒
No-PTZ version	☐
Required peripheral	ESP32 camera (bracket-mounted), and a printed AprilTag marker (tag36h11 family)

2. Start the Agent + Camera + Joint Model (Terminal 1)

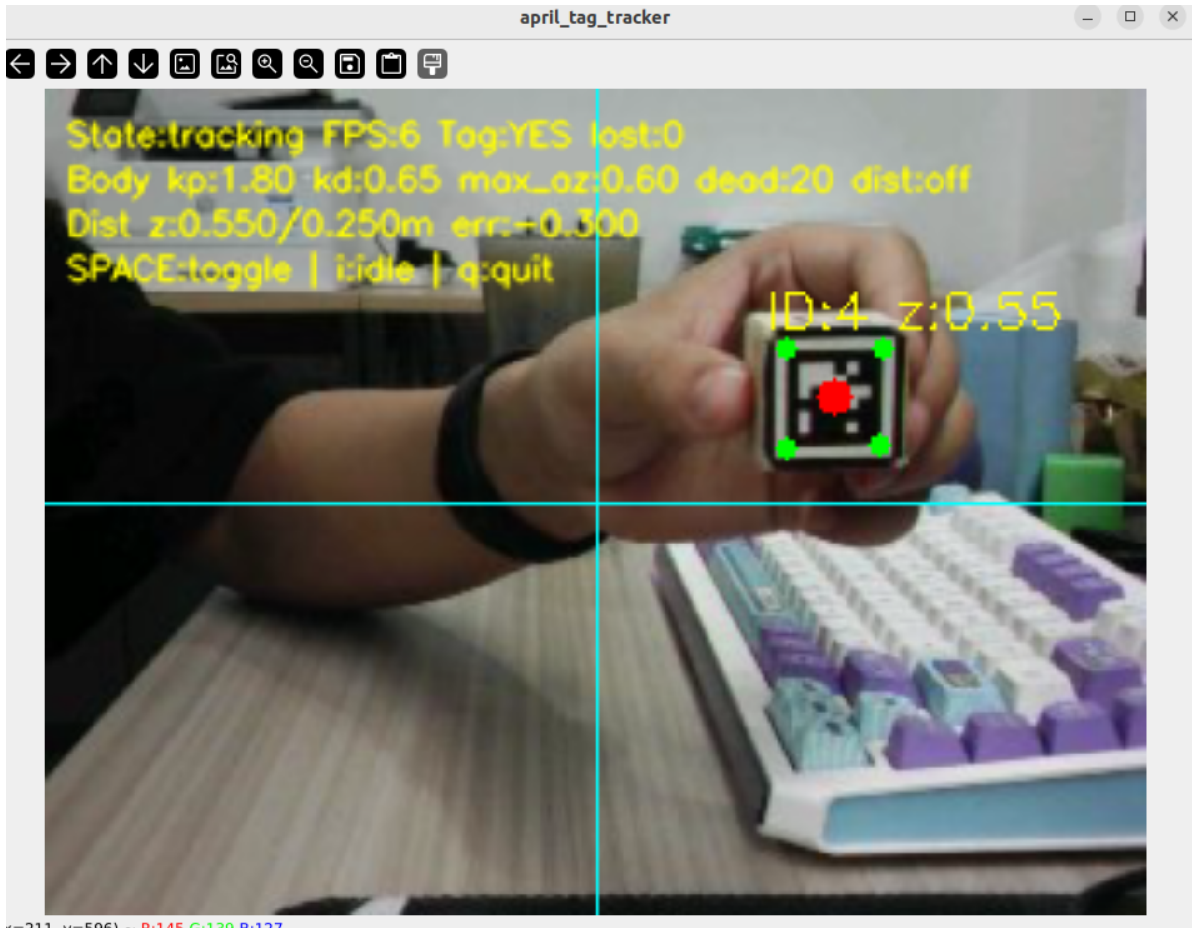
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom: $ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start AprilTag Tracking (Terminal 2)

Tracking mode (rotation tracking only):

```
ros2 launch microros_v2_no_ptz_visual_control_pkg april_tag_tracker.launch.py
```



5. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

- When the AprilTag moves left or right in the frame, the car automatically rotates to track it, keeping the Tag at the center of the frame
- The terminal outputs the Tag's 3D pose information in real time (translation x/y/z and rotation angles)
- The Tag border and center point are visualized
- Follow mode: keeps a constant distance based on the Tag depth Z value
- After the Tag moves out of the frame, the car stops; when it reappears, tracking resumes automatically

5. Code Explanation

Source path:

- Launch (track):

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/april_tag_tracker.launch.py
```

- Launch (follow):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/april_tag_follow.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py`

1. Core Node Analysis

`april_tag_tracker_node` uses the `apriltag` Python library to detect AprilTag markers. Core workflow:

- **Image processing** (15Hz): raw Image frame conversion → grayscale → `detector.detect(gray)` → extract Tag corners + pose
- **3D pose estimation**: uses the PnP algorithm with the known physical Tag size (`tag_size`) and camera intrinsics to compute the 6-DoF pose
- **Body control** (10Hz): deviation between the Tag center x-coordinate and the image center → PD angular velocity; Tag depth Z-value deviation → PD linear velocity
- `quad_decimate`: lowers the detection resolution to speed up processing (the larger the value, the faster, but small Tags at long distance may be missed)

Constructor — create the AprilTag detector:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._tgt_lock = threading.Lock()
    self._state = STATE_IDLE
    self._has_target = False
    self._lost_frames = 0
    self._target_x: Optional[float] = None
    self._target_z: Optional[float] = None # Tag depth Z value (used for
distance following)

    self._declare_params()
    self._load_params()

    # Create the AprilTag detector
    self._detector = apriltag.Detector(
        apriltag.DetectorOptions(
            families=self._tag_family,
            quad_decimate=float(self._quad_decimate),
        )
    )

    best_effort = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, best_effort)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, best_effort)
```

```

        self._img_timer = self.create_timer(
            1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
        self._body_timer = self.create_timer(
            1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ),
self._body_tick)

```

Body control callback — 3D Z-value distance following:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/april_tag_tracker_node.py

def _body_tick(self) -> None:
    """Body control timer (10Hz)."""
    if self._state != STATE_TRACKING or self._shutdown:
        self._pub_zero()
        return

    with self._tgt_lock:
        has = self._has_target
        tx = self._target_x
        tz = self._target_z

    if not has or tx is None:
        self._pub_zero()
        return

    # Horizontal pixel error → PD angular velocity
    cx = self._img_w * 0.5
    err_x = tx - cx
    if abs(err_x) <= self._deadband_x:
        az = 0.0
    else:
        az = self._angular_kp * err_x
        az = clamp(az, -self._max_az, self._max_az)
        if self._reverse_body:
            az = -az

    cmd = Twist()
    cmd.angular.z = az

    # Distance following: based on the 3D depth Z value (not pixel size)
    if self._dist_follow and tz is not None and tz > 0:
        err_z = self._target_z_m - tz # _target_z_m unit: meters
        if abs(err_z) > self._dist_tol_z:
            lx = self._dist_kp * err_z
            lx = clamp(lx, -self._max_lx, self._max_lx)
            if self._reverse_linear:
                lx = -lx
            cmd.linear.x = lx

    self._cmd_pub.publish(cmd)

```

Key logic: Unlike the pixel-size-based distance following in color/face tracking, AprilTag distance following directly uses the 3D pose Z value (actual depth), so the actual size of the target does not need to be calibrated. The `tag_size` parameter must match the actual physical side length of the printed Tag (default 0.048 m); otherwise the pose estimation will be inaccurate.

2. Key Parameters

Parameter definition file:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/april_tag_tracker.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/april_tag_tracker_node.py`

Parameter	Type	Default	Description
<code>tag_family</code>	str	<code>tag36h11</code>	AprilTag marker family
<code>tag_size</code>	float	<code>0.048</code> m	Physical side length of the Tag (meters), affects the 3D pose computation accuracy
<code>quad_decimate</code>	float	<code>2.0</code>	Image downsampling factor
<code>tag_select_strategy</code>	str	<code>largest</code>	Multi-Tag selection strategy
<code>deadband_x_pixel</code>	float	<code>20.0</code> px	Horizontal deadband
<code>angular_kp</code>	float	<code>1.8</code>	Body angle PD proportional coefficient
<code>max_angular_z</code>	float	<code>0.6</code> rad/s	Maximum body steering speed
<code>enable_distance_follow</code>	bool	<code>false</code>	Whether to enable distance following
<code>target_tag_pose_z</code>	float	<code>0.25</code> m	Target Tag depth for distance following (3D Z value)
<code>distance_tolerance_pose_z</code>	float	<code>0.02</code> m	Distance-following deadband
<code>distance_kp</code>	float	<code>1.5</code>	Distance PD proportional coefficient
<code>max_linear_x</code>	float	<code>0.30</code> m/s	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>april_tag_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	AprilTag 6-DoF tracking with 3D pose + PD body control

6. Common Problems

Problem	Cause	Solution
Tag is not detected	Tag too small, reflective, or <code>quad_decimate</code> too large	Enlarge the printed Tag size and lower <code>quad_decimate</code>
Slow steering tracking	PD parameters too conservative	Increase <code>angular_kp</code> and reduce <code>deadband_x_pixel</code>
Pose jitter	The physical Tag size <code>tag_size</code> is inaccurate	Precisely measure and set the actual side length of the Tag (meters)
Slow Follow mode response	Distance PD parameters are unreasonable	Tune <code>distance_kp</code> / <code>distance_kd</code>